

RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Project Report
**Repository Explainer: An Intelligent RAG-oriented
Web Application with AI-Agentive Capability
for Understanding GitHub Repositories**

Course Code : CSE 3100

Course Name : Web Based Application Project

<u>Submitted by:</u> Name : Tamjid Hasan Fahim Roll : 2103052 Section : A Department: Computer Science and Engineering Rajshahi University of Engineering and Technology	<u>Submitted to:</u> Utsha Das Assistant Professor Department of Computer Science and Engineering Rajshahi University of Engineering and Technology
--	--

Date of Submission : 23-08-2025

CONTENTS

Project Source Code.....	3
1 Introduction.....	4
1.1 Problem Statement.....	4
1.2 Goal of the project.....	4
2 Existing Works.....	4
2.1 The Limitation of Large Language Models.....	5
3 Novelty of this Project.....	5
3.1 RAG System & Embeddings.....	5
3.2 Dual-Granularity Indexing.....	6
3.3 Query-Aware Retrieval Strategy.....	6
3.4 Agentic Lookup Loop.....	7
3.5 Language-Agnostic Logical Unit Extraction.....	7
4 Tech Stack.....	8
5 Methodology.....	8
5.1 The High-Level Summary Generation Pipeline.....	10
5.1.1 Intelligent File Categorization.....	11
5.1.2 Context Aggregation.....	11
5.1.3 Structured Documentation Generation.....	11
5.1.4 Caching for Efficiency.....	11
5.2 The Indexing Pipeline.....	12
5.2.1 Cloning and Filtering.....	13
5.2.2 Code Unit Extraction.....	13
5.2.3 Dual-Granularity Chunking.....	13
5.2.4 Embedding and Storing in Vector DB.....	13
5.2.5 Repository Map Generation.....	14
5.3 The Question-Answering (RAG) Pipeline.....	15
5.3.1 Query Classification.....	16
5.3.2 Retrieval Strategy.....	16
5.3.3 Agentic Lookup Loop.....	16
5.3.4 Answer Generation.....	16
6 Project Testing.....	17

7	Conclusion	17
7.1	Future Work.....	17
8	References.....	18
9	Appendix.....	20
9.1	Project Screenshots of Summary Generation Interface	20
9.2	Project Screenshots of Question-Answering Interface	25
9.3	Project Screenshots of Backend Console	29

PROJECT SOURCE CODE

The source code for this project is available at the following GitHub repository:

<https://github.com/heytaamjid/repositoryExplainer>

1 INTRODUCTION

Navigating and understanding large, unfamiliar software repositories is a significant challenge for developers. Whether onboarding to a new team, contributing to an open-source project, or conducting a code audit, developers often spend hours, if not days, manually piecing together the architecture, core logic and data flow of a codebase. This process is time-consuming, inefficient, and prone to misunderstanding.

1.1 PROBLEM STATEMENT

This project addresses the following key problems:

- **Cognitive Overload:** Modern codebases can contain thousands of files and millions of lines of code, making it difficult for developers to build a mental model of the system.
- **Inefficient Knowledge Discovery:** Finding specific implementation details or understanding high-level architectural patterns often involves tedious manual searching and code tracing.
- **Onboarding Friction:** New developers or open-source enthusiasts face a steep learning curve when trying to become productive in a new repository.

1.2 GOAL OF THE PROJECT

The goal of this project is to develop an intelligent system that can analyze a GitHub repository, understand its structure, content & functionalities, provide a high-level summary of the codebase and answer developer's questions in natural language. The system aims to serve as a senior "*code guide*", providing both high-level summaries as well as detailed answers about the codebase.

2 EXISTING WORKS

Several tools and techniques exist to aid code understanding. Traditional methods include:

- **Code Search Engines:** Tools like Sourcegraph and GitHub's native search allow for powerful text and regex-based searches across a codebase, but they require the user to know what to search for.
- **Documentation Generators:** Tools like Sphinx or Javadoc generate documentation from code comments, which is helpful but relies on the quality and completeness of the comments themselves.

- **AI-powered tools:** More recently, tools like GitHub Copilot and Amazon CodeWhisperer have emerged. While these are excellent for code generation and completion, their primary focus is on the local context of the file being edited rather than the global context of the entire system.

2.1 THE LIMITATION OF LARGE LANGUAGE MODELS

The core issue is that LLMs process information within a finite "context window." This window acts like a short-term memory, only allowing the model to focus on a limited number of tokens (words or pieces of words) at a time.

A small, self-contained code snippet easily fits within this context window, allowing the LLM to analyze it and provide a coherent explanation. However, for an entire codebase, which can contain thousands or millions of lines of code across multiple files, is far too large to fit into the context window. The model cannot "see" the entire project at once. It loses track of how different files and functions interact, which is crucial for understanding a large system.

Furthermore, when an LLM is asked about something outside its immediate context, it often resorts to hallucination. This means it generates plausible-sounding but factually incorrect information because it lacks the necessary data to provide an accurate answer. This tendency is a major risk when trying to explain a complex system like an entire codebase, where the model must understand deep, interconnected relationships it cannot fully perceive.

3 NOVELTY OF THIS PROJECT

This project introduces a multi-layered Retrieval-Augmented Generation (RAG) framework with AI-agent tailored for code understanding, designed to overcome the limitations of large language models (LLMs) when dealing with complex, large-scale repositories. Instead of relying solely on an LLM's pre-trained memory, our approach retrieves and integrates relevant context from the codebase itself, ensuring responses are both precise and grounded in the actual repository.

3.1 RAG SYSTEM & EMBEDDINGS

At a high level, the RAG workflow consists of:

- **Retrieval:** Locating the most relevant files or code units in response to a query.
- **Augmentation:** Supplying this retrieved information alongside the query to the LLM.
- **Generation:** Producing an answer that is accurate, contextual, and repository-specific.

In RAG systems, **embeddings** are the foundation for retrieving meaningful context for the LLM. Embeddings are numerical vector representations of text or code that capture their semantic meaning. By converting functions, files, or documentation into vectors in a high-dimensional space, semantically similar items are placed closer together. This allows efficient similarity search without any strict keyword or even fuzzy string matching. When a query is embedded, the system retrieves the most relevant code or documents (most *semantically close* ones) by comparing distances in this vector space.

What makes our solution novel is not just the application of RAG, but the layered design and adaptive mechanisms that ensure retrieval is context-sensitive, efficient and aligned with how human developers explore unfamiliar code. Key unique techniques adopted in the projects are discussed below.

3.2 DUAL-GRANULARITY INDEXING

We create two levels of embeddings:

- File-level embeddings capture high-level ideas such as architecture, modules, and design choices.
- Logical Unit-level (functions, classes) embeddings capture details at the scale of functions, classes, or blocks of code.

File-level embeddings capture high-level architectural meaning but are too coarse for fine-grained implementation details. Conversely, unit-level embeddings of functions and classes provide precision but lose broader structural context, making it difficult to answer design-level questions. This creates a precision–context dilemma: retrieving narrowly risks incomplete answers, while retrieving broadly introduces noise and overwhelms the LLM’s limited context window.

Developers often need to reason at different levels of abstraction. Sometimes the question is broad (“What is the overall purpose of this repo?”), and other times it is specific (“How does this function validate user input?”). A single granularity cannot effectively support both. By maintaining dual-granularity indexing, we ensure the system can retrieve the right level of context for the type of question asked.

3.3 QUERY-AWARE RETRIEVAL STRATEGY

The system begins by classifying the user’s question as either:

- High-level → triggers a two-step retrieval: first relevant files, then logical units within those files.
- Low-level → directly retrieves the most relevant logical code units.

This mirrors how developers themselves approach codebases. A high-level design question requires first locating the architectural components, whereas a low-level implementation question

needs a direct jump into specific code units. By dynamically adapting the retrieval strategy to the query type, the system avoids wasting tokens on irrelevant details and keeps the context precisely scoped.

3.4 AGENTIC LOOKUP LOOP

Beyond one-time retrieval, the system introduces an agentic loop:

- After the first retrieval, a planner model evaluates whether the context is sufficient.
- If gaps remain, it generates follow-up lookups (e.g., “fetch the full content of this file” or “load this specific function’s code”).
- These lookups enrich the context until the system has enough to confidently answer.

A static retrieval pass is often insufficient for deep or multi-step questions. Human developers rarely stop after one glance; they iteratively explore files and dependencies until they gather enough information. Our agentic loop mimics this exploration, enabling dynamic and on-demand expansion of the context window.

3.5 LANGUAGE-AGNOSTIC LOGICAL UNIT EXTRACTION

The system uses:

- Abstract Syntax Trees (ASTs) for Python, ensuring highly precise extraction of functions and classes.
- Language-aware text splitters for other languages like JavaScript, Java, Go, and Rust, allowing broader coverage.

Repositories are rarely monolingual. A robust code-understanding system must work across ecosystems. While AST parsing provides the highest accuracy, building custom parsers for every language is a time-consuming ongoing work for this project. By combining AST precision where possible with generalizable splitters elsewhere, our system achieves both adaptability and scalability, ensuring it can handle diverse, polyglot repositories.

We employ logical unit-level chunking (functions, classes, or coherent code blocks) instead of simple flat chunking because it preserves the semantic and structural integrity of the code. Flat chunking may split logical units across boundaries, making embeddings less meaningful and retrieval less precise, whereas logical-level chunks align with actual code semantics, enabling more accurate context retrieval for answering detailed implementation questions.

4 TECH STACK

The project is built on a modern stack of Python libraries and frameworks, designed for building AI-powered web applications.

- **Backend Framework: Django** is used to build the web application, providing views, API endpoints and template rendering.
- **Large Language Models (LLMs):** The project relies on Google's Gemini models, accessed via the langchain-google-genai library. Specifically, "gemini-2.5-flash" is used for its speed and capability in tasks like file identification, documentation generation, and question answering.
- **LLM Orchestration: LangChain** is used to structure the interactions with the LLMs. It provides tools for creating prompts (ChatPromptTemplate), chaining operations, and parsing outputs (StrOutputParser).
- **Embedding Models:** The system supports both local and remote embedding models.
 - **Remote:** Google's models/embedding-001 for high-quality embeddings via an API call.
 - **Local:** nomic-ai/nomic-embed-text-v1.5, run using the sentence-transformers library from HuggingFace, for users who prefer to run the pipeline locally.
- **Vector Database: ChromaDB** serves as the vector store. It efficiently stores the embeddings of the code chunks and allows for fast similarity searches, which is the core of the retrieval mechanism.
- **Code Parsing & Splitting:**
 - Python's built-in ast module is used for precise extraction of functions and classes from Python files.
 - langchain_text_splitters provides RecursiveCharacterTextSplitter with language-specific separators to intelligently chunk other source code files.
- **GitHub Integration:** The requests library is used to interact with the GitHub API for fetching repository trees and file contents.

5 METHODOLOGY

The project's methodology can be broken down into three main pipelines: the High-Level Summary Generation Pipeline (generates the high-level summary of the codebase), Indexing

Pipeline (processing and storing the codebase) and the Question-Answering Pipeline (retrieving information and generating answers).

5.1 THE HIGH-LEVEL SUMMARY GENERATION PIPELINE

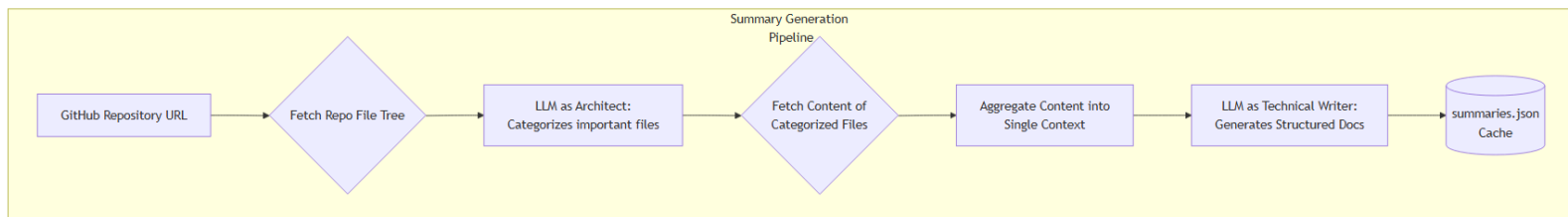


Figure 1: High-level Summary Generation Pipeline

Before any interactive questioning can occur, the system performs a one-time, high-level analysis to generate a structured summary of the repository. This process, primarily orchestrated by the home view in `views.py`, provides users with an immediate, readable overview, similar to a project's README, but generated entirely from the source code itself. The high-level summary is also passed to the LLM to give it the bigger picture of the codebase while answering user's questions.

5.1.1 Intelligent File Categorization

The process begins by fetching the entire tree of file paths from the repository. Instead of analyzing every file, the system uses an LLM as an expert software architect. The `get_important_files_by_category` function passes the complete list of files to the model. The model's task is to return a JSON object that maps the most relevant file paths to predefined, high-level categories such as "*Purpose & Scope*," "*System Architecture Overview*," "*Core Components & Business Logic*," and "*Data Flow*". This step intelligently filters the repository down to only the most critical files for understanding its core concepts.

5.1.2 Context Aggregation

Once the most important files are categorized, the `generate_documentation` function fetches the full content of these files from GitHub. The content of each relevant file is concatenated into a single, large context string, prefixed with a header indicating the file's path.

5.1.3 Structured Documentation Generation

This aggregated context is then sent to the LLM again, which is prompted with a "technical writer" persona. The model is instructed to generate a high-level, structured documentation for the mentioned sections based *only* on the provided file contents.

5.1.4 Caching for Efficiency

The final generated documentation, along with the latest commit hash of the repository, is saved to a local JSON file (`summaries.json`). On subsequent visits to the application for the same repository, this cached summary is displayed instantly, avoiding the need to re-run this computationally intensive generation process unless the user explicitly forces a re-summarization.

5.2 THE INDEXING PIPELINE

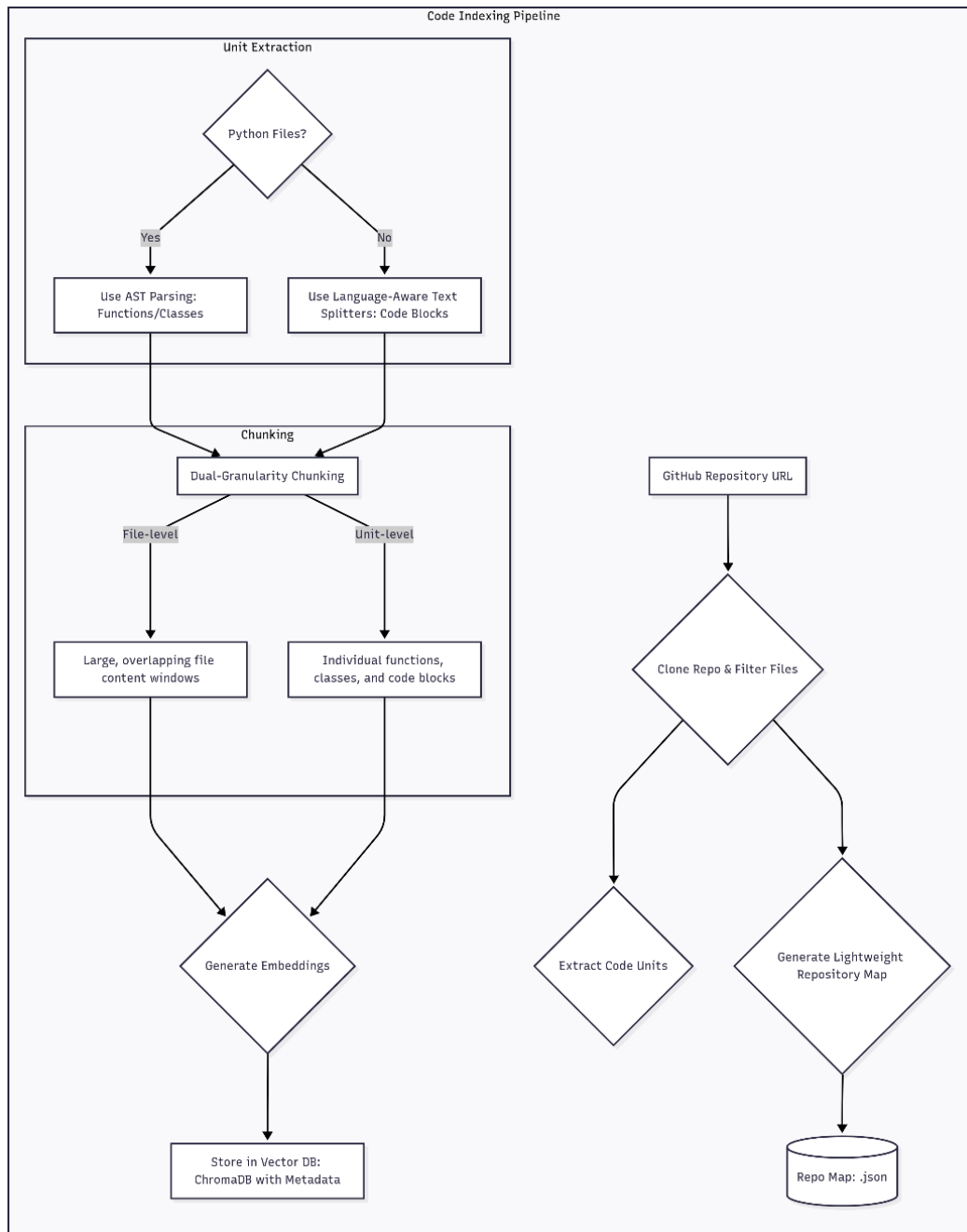


Figure 2: Chunking and Embedding Pipeline

This pipeline is responsible for ingesting a GitHub repository into embedding vectors and preparing it for querying. This process makes the codebase semantically searchable. The logic for this is primarily located in *embedder.py*.

5.2.1 Cloning and Filtering

The system begins by cloning the target GitHub repository to a temporary directory. It then intelligently filters out irrelevant files by respecting the repository's .gitignore rules and applying a built-in denylist of common noisy directories, boiler-plate files (i.e. node_modules, venv, static bundle files) and binary file extensions (.png, .zip, .sqlite3 etc.). This ensures that only meaningful source code and documentation are indexed.

5.2.2 Code Unit Extraction

For each valid file, the system extracts *logical units* of code.

For Python files (.py): It uses the ast module to parse the file into an Abstract Syntax Tree and reliably extracts every class and function, including its name, line numbers, and full source code.

For other languages: It uses RecursiveCharacterTextSplitter from LangChain, configured with separators common to many programming languages (e.g., class, function, \n\n). This allows it to break down non-Python code into semantically meaningful chunks and keeps the logical units unified in the embedding representation.

5.2.3 Dual-Granularity Chunking

This is a crucial step for enabling nuanced retrieval.

Unit-Level Indexing: Each extracted unit (function, class or generic code block) is treated as a separate document. These documents are embedded and stored in ChromaDB with a granularity metadata tag set to "unit". This index is used for answering low-level, implementation-specific questions.

File-Level Indexing: The entire content of larger files is also chunked into larger, overlapping windows. These chunks are embedded and stored with a granularity metadata of "file". This index captures the broader context of a file and is used for high-level, architectural questions.

Both kinds of indexing stores enough metadata (i.e. file path, line range etc.) to trace back a code snippet in the original codebase.

5.2.4 Embedding and Storing in Vector DB

All the generated embedded chunks (both unit-level and file-level) are converted into vector embeddings via either a remote model (models/embedding-001) or a local model (nomic-ai/nomic-embed-text-v1.5). User can pick any of the two options based on their convenience. Both of the embedding models are decent enough and trained on code-to-text and text-to-code data. These embeddings, along with their rich metadata (file path, unit name, line numbers, granularity), are stored in a ChromaDB collection.

5.2.5 Repository Map Generation

Finally, a lightweight JSON "map" of the repository is created and saved. This map lists all processed files and the names & locations of the units within them, but without the full code content. This map is used later by the agentic planner to identify potential lookup targets.

5.3 THE QUESTION-ANSWERING (RAG) PIPELINE

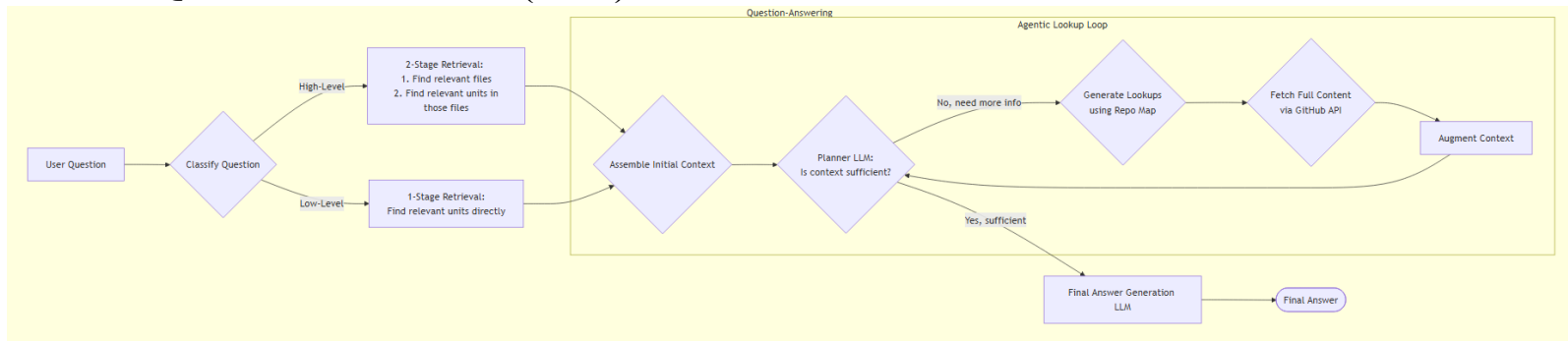


Figure 3: Question-Answering Pipeline

When a user asks a question, this pipeline, primarily defined in the `api_ask` function in `views.py`, is activated.

5.3.1 Query Classification

The user's question is first classified as either "*high-level*" or "*low-level*" using a very lightweight local sentence-transformer model (all-MiniLM-L12-v2) from HuggingFace. This initial step determines the entire retrieval strategy that follows.

5.3.2 Retrieval Strategy

If the question is high-level (e.g., "What is the overall data flow of this process?"), the system performs a **two-stage retrieval**. First, it queries the file-level index to find the most relevant *files*. Then, for each of those top files, it performs a second query against the unit-level index to retrieve the most relevant *units* from *within* those files. This ensures that the context contains both broad overviews and key supporting details.

If the question is low-level (e.g., "How is this feature implemented?"), the system performs a **single-stage retrieval**, querying the unit-level index directly to find the most relevant functions or classes across the entire codebase.

5.3.3 Agentic Lookup Loop

After the initial context is retrieved, the system enters a loop to decide if more information is needed.

Plan: An LLM planner is given the user's question, the currently retrieved context and the repository map we saved. It is asked to decide if lookups for additional source code are needed to answer the question confidently. If so, it must output a structured JSON object specifying which files or functions to fetch.

Act: If the planner requests lookups, the system uses the GitHub API to fetch the full content of the requested items.

Augment: The newly fetched content is appended to the existing context. The loop can run multiple times (currently capped at 2 iterations) to progressively build a comprehensive context.

5.3.4 Answer Generation

The final, augmented context is passed to the LLM (gemini-2.5-flash) via API endpoints. This model is prompted to synthesize all the provided information -- retrieved code chunks, on-demand lookups snippets, and repository summaries -- to generate a comprehensive, human-readable answer to the user's original question.

6 PROJECT TESTING

We evaluated our developed web application, Repository Explainer, on a variety of GitHub repositories, including the repository of this project itself, to assess its performance and robustness. The system performed effectively across different types of repositories, accurately generating high-level summaries and answering queries. Some representative screenshots demonstrating its functionality and interface are given in the Appendix section.

7 CONCLUSION

This project successfully demonstrates the power of combining Retrieval-Augmented Generation (RAG), dual-granularity indexing and agentic workflows to create a powerful tool for repository understanding. By mimicking the exploratory process of a human developer, the system can provide relevant and context-rich answers to a wide range of questions, significantly reducing the cognitive load and time required to understand a complex codebase.

7.1 FUTURE WORK

In future work, we plan to extend this system by integrating more advanced retrieval and reasoning mechanisms to further enhance its capacity for repository understanding. A key direction is the expansion of hierarchical indexing beyond the current dual-granularity design into multi-level structures (repository $\rightarrow$ package $\rightarrow$ file $\rightarrow$ function), thereby supporting efficient drill-down and roll-up navigation across different abstraction levels across multiple repositories (i.e. frontend, backend and microservices repository). Incorporating Hypothetical Document Embeddings (HyDE) will allow the system to generate and embed hypothetical answers, improving retrieval performance particularly for underspecified or ambiguous queries. Similarly, the adoption of RAPTOR-style recursive summarization can provide multi-resolution representations of repositories, enabling coherent, layered exploration of both high-level architecture and low-level implementation. Additionally, step-back prompting and query decomposition techniques can support the answering of complex, multi-stage questions by guiding retrieval towards progressively refined contexts. Expanding AST-based parsing to cover more programming languages will improve precision in polyglot repositories. Additionally, the agent can be extended with a keyword-based file or code snippet search capability, allowing it to directly locate files or functions by name or identifier. Combining these more advanced techniques will significantly enhance the system’s ability to provide accurate, context-aware, and comprehensive answers across diverse and complex codebases, reduce developer cognitive load, and enable efficient exploration of multi-repository architectures while maintaining scalability and adaptability.

8 REFERENCES

1. **Retrieval-Augmented Generation (RAG):** For the core RAG framework that combines retrieval with generation to ground LLM responses in external knowledge.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems, 33, 9459-9474.
2. **Agentic Workflows (ReAct Framework):** For the "plan-act" loop in your agentic lookup, which mirrors the reasoning and action steps described in the ReAct paper.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv preprint arXiv:2210.03629.
3. **Sentence Embeddings (Sentence-BERT):** For the use of sentence-transformer models like *all-MiniLM-L12-v2* to create semantic embeddings for text.

Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing.
4. **Nomic Embed:** Open-source code-text embedding model used in the project.

Nomic AI. (2024). *nomic-embed-text-v1.5*. Hugging Face. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>
5. **LLM Orchestration (LangChain):** For the framework used to structure interactions with LLMs, chain operations, and manage prompts.

Chase, H. (2022). *LangChain*. GitHub. <https://github.com/langchain-ai/langchain>
6. **Vector Database (ChromaDB):** For the vector store used to efficiently store and search code embeddings.

Chroma. (2023). *Chroma*. GitHub. <https://github.com/chroma-core/chroma>
7. **Hypothetical Document Embeddings (HyDE):** For the future work idea of generating a hypothetical answer to improve retrieval.

Gao, L., Ma, X., Lin, J., & Callan, J. (2022). *Precise Zero-Shot Dense Retrieval without Relevance Labels*. arXiv preprint arXiv:2212.10496.
8. **Recursive Summarization (RAPTOR):** For the future work concept of building a multi-resolution index of the repository through recursive clustering and summarization.

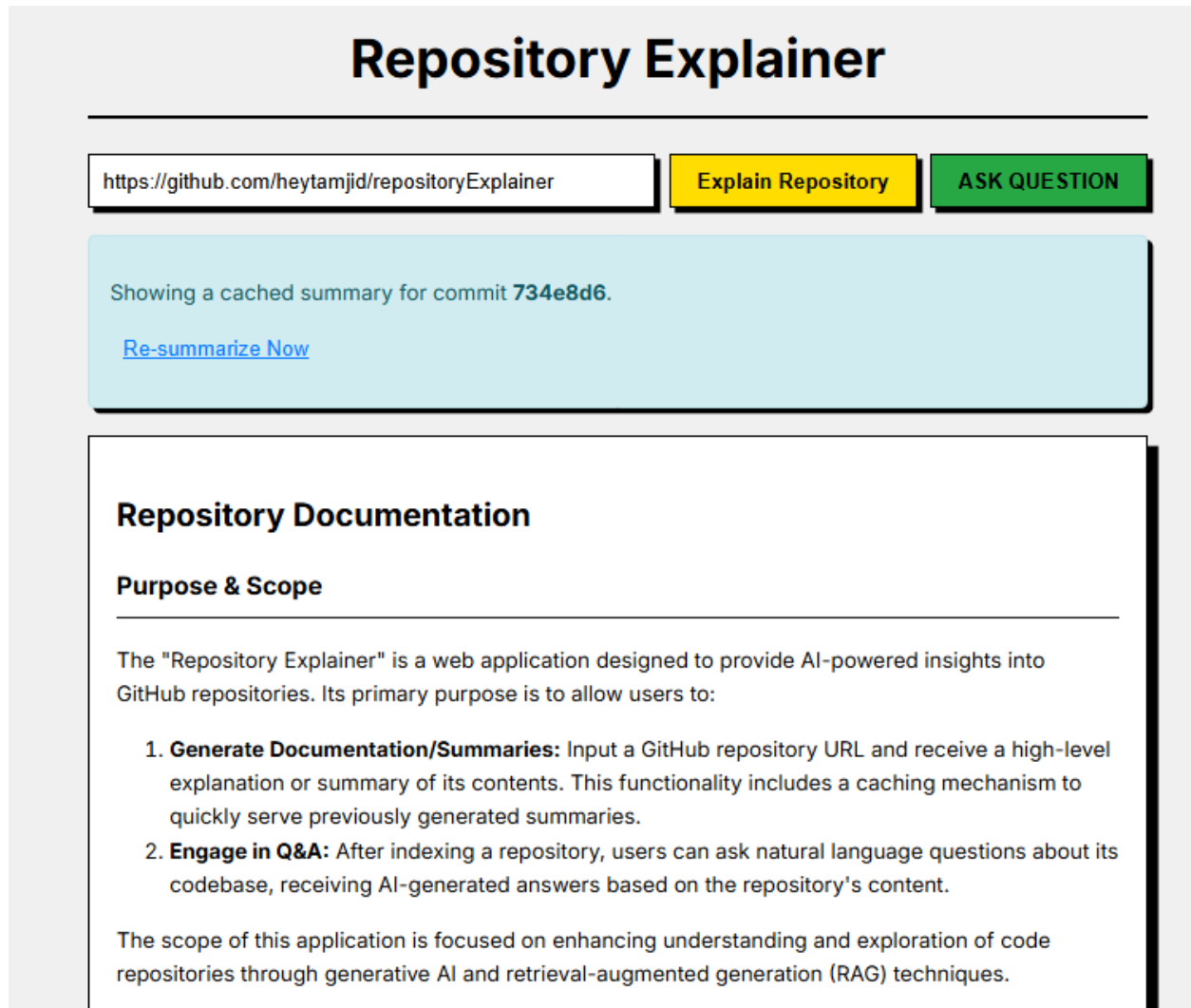
Sarathi, P., et al. (2024). *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval*. arXiv preprint arXiv:2401.18059.

9. **Step-Back Prompting:** For the future work technique of abstracting away from specific details to generate a more general query, improving retrieval for complex questions.

Zheng, H., et al. (2023). *Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models*. arXiv preprint arXiv:2310.06117.

9 APPENDIX

9.1 PROJECT SCREENSHOTS OF SUMMARY GENERATION INTERFACE



System Architecture Overview

The system is built as a Python-based web application using the Django framework, incorporating advanced AI/ML capabilities.

- **Frontend:** The user interface is rendered using Django's templating engine, primarily `home.html` for repository summarization and `ask.html` for the interactive Q&A chat. Client-side JavaScript handles dynamic interactions, such as initiating indexing and sending/receiving chat messages.
- **Backend:** A Django application serves as the core backend. It handles HTTP requests, manages application settings, and orchestrates the complex AI/ML workflows. It exposes both traditional web views (for rendering HTML pages) and API endpoints (for client-side JavaScript interactions).
- **Database:** The application uses SQLite (`db.sqlite3`) as its default database, primarily for Django's internal functionalities such as user authentication, sessions, and administrative data. There are no custom models defined within the `myapp` application, suggesting that application-specific data (like generated summaries or chat history) might be handled differently or not persisted in this database.
- **AI/ML & Data Processing Components:**
 - **Langchain:** Acts as an orchestration framework for integrating various language models and tools.
 - **Google Generative AI:** Utilizes Google's large language models for generating explanations, summaries, and answers.
 - **ChromaDB:** A vector database used to store embeddings of the repository's code and documentation, enabling efficient semantic search for the Q&A feature.
 - **PyGithub:** Facilitates interaction with the GitHub API to fetch repository contents.
 - **Text Processing & Embeddings:** Libraries like `langchain-text-splitters`, `sentence-transformers`, `torch`, and `transformers` are used for breaking down repository content into manageable chunks and converting them into numerical vector embeddings.
- **Caching Mechanism:** The system incorporates a caching layer for repository summaries, indicated by the "Showing a cached summary" feature in `home.html`, to improve performance and reduce redundant AI processing.
- **Environment Configuration:** Utilizes `python-dotenv` to manage sensitive configuration, such as the Django `SECRET_KEY`, through environment variables.

Core Components & Business Logic

The application's core functionality is distributed across its Django components and integrated AI/ML libraries:

- **Django Project (`repositoryExplainer`):**
 - `settings.py`: Configures the entire Django project, including `myapp` as an installed application, SQLite database, template paths, and static file handling. It also loads environment variables for sensitive data.
 - `urls.py`: Defines the top-level URL routing, directing requests to the Django admin interface and the `myapp` application.
 - `asgi.py` / `wsgi.py`: Standard entry points for asynchronous and synchronous web server gateways.
- **`myapp` Application:**
 - `urls.py`: Defines specific routes for the application:
 - `/`: Maps to the `home` view for repository summarization.
 - `/ask/`: Maps to the `ask_question` view for the Q&A interface.
 - `/api/ask/`: An API endpoint for handling AJAX requests related to indexing and asking questions.
 - **Views (inferred `views.py`):**
 - **`home` view:** Responsible for rendering `home.html`. It processes submitted GitHub URLs, triggers repository summarization (potentially checking cache first), and displays the generated documentation. It also handles requests to force re-summarization.
 - **`ask_question` view:** Renders `ask.html`, setting up the client-side chat interface.
 - **`api_ask` view:** This is the central hub for AI interaction. It handles two main actions:
 - **`start_indexing`:** Fetches repository content using `PyGithub`, processes it (e.g., using `langchain-text-splitters`), generates embeddings, and stores them in `ChromaDB`.
 - **`ask`:** Takes a user question, embeds it, retrieves relevant context from `ChromaDB`, constructs a prompt, and queries `Google Generative AI` (orchestrated by `Langchain`) to generate an answer.
 - **Templates (`home.html`, `ask.html`):** Provide the user-facing forms and display areas. `home.html` displays the generated repository documentation, while `ask.html` provides an interactive chat interface.
 - **Template Tags (`custom_filters.py`):** Contains utility functions like `get_item` to access dictionary values within Django templates.

access dictionary values within Django templates.

- **AI/ML Business Logic:**

- **Repository Content Acquisition:** `PyGithub` is used to programmatically access and download files from specified GitHub repositories.
- **Text Chunking:** Repository files are likely broken down into smaller, manageable chunks suitable for embedding and LLM context windows using `langchain-text-splitters`.
- **Embedding Generation:** `sentence-transformers` or similar models generate vector representations (embeddings) of these text chunks and user questions.
- **Vector Storage and Retrieval:** `ChromaDB` stores the generated embeddings. During Q&A, it performs similarity searches to retrieve the most relevant code/text chunks for a given question.
- **Generative AI:** `Langchain` facilitates sending prompts (composed of user questions and retrieved context) to `Google Generative AI` to synthesize coherent explanations or answers.
- **Caching:** A mechanism (details not fully specified but inferred from `home.html`) stores and retrieves generated repository summaries to avoid reprocessing.

Data Flow

1. Initial Page Load (`home.html`):

- A user navigates to the root URL (`/`).
- The Django `home` view renders `home.html`.

2. Repository Explanation Flow:

- **Input:** User enters a GitHub repository URL into the input field on `home.html` and clicks "Explain Repository".
- **Request:** A POST request is sent to the `home` view with the `repo_url`.
- **Caching Check:** The backend checks if a summary for the given `repo_url` (and potentially commit hash) exists in its cache.
- **Processing (if not cached or forced):**
 - `PyGithub` fetches files from the specified GitHub repository.
 - The content is processed (e.g., parsed, chunked).
 - `Langchain` orchestrates calls to `Google Generative AI` to generate a structured summary/documentation based on the repository content.
 - The generated documentation is stored in the cache.
- **Output:** The `home` view renders `home.html` again, displaying the generated `documentation` (or cached version) within the `documentation-container`.

3. Q&A Interface Navigation:

- User clicks "ASK QUESTION" on `home.html` or directly navigates to `/ask/`.
- The `ask_question` view renders `ask.html`. The `repo_url` might be pre-filled from a query parameter.

4. Repository Indexing Flow (for Q&A):

- **Input:** User enters a GitHub repository URL into the `repoUrl` field on `ask.html` and clicks "Index Repository".
- **Request:** Client-side JavaScript sends an AJAX POST request to `/api/ask/` with `action: 'start_indexing'` and `repo_url`.
- **Processing:**
 - The `api_ask` view receives the request.
 - `PyGithub` fetches repository content.
 - `langchain-text-splitters` break down the content into chunks.
 - `sentence-transformers` (or similar) generate vector embeddings for these chunks.
 - These embeddings, along with their original text, are stored in `ChromaDB`.
- **Response:** The `api_ask` view returns a JSON response indicating `indexing_started`.
- **UI Update:** The client-side JavaScript updates the chat interface to show that indexing has begun.

5. Question Answering Flow:

- **Input:** User types a `question` into the `userInput` field on `ask.html` and clicks "Send".
- **Request:** Client-side JavaScript sends an AJAX POST request to `/api/ask/` with `action: 'ask'`, `question`, and the current `repo_url`.
- **Processing:**
 - The `api_ask` view receives the request.
 - The user's `question` is embedded into a vector.
 - A similarity search is performed in `ChromaDB` (scoped to the indexed `repo_url`) to retrieve the most relevant code/text chunks.
 - `Langchain` constructs a prompt using the `question` and the retrieved context.
 - This prompt is sent to `Google Generative AI`.
 - `Google Generative AI` generates an `answer`.
- **Response:** The `api_ask` view returns a JSON response containing the `answer`.
- **UI Update:** The client-side JavaScript receives the `answer`, renders it as Markdown using `marked.parse`, and appends it to the chat `messages` display.

9.2 PROJECT SCREENSHOTS OF QUESTION-ANSWERING INTERFACE

Ask Questions about the Repository

First, click the "Index Repository" button to make the codebase searchable. Then, you can ask natural-language questions.

Repository:

Index Repository

Hi! Please index a repository to begin.

Starting to index https://github.com/heytaamjid/repositoryExplainer... This may take a few minutes.

Indexing process has begun on the server. You can start asking questions once it is complete.

You: how does the dual level chunking work?

The dual-level chunking mechanism involves creating and utilizing two distinct granularities of code chunks: **file-level chunks** and **unit-level chunks**. This approach is used both during the indexing of a repository and during advanced retrieval.

1. Indexing (Creation of Dual-Level Chunks)

The `index_repository` function in `myapp/embedder.py` is responsible for creating these chunks and embedding them into a vector database (ChromaDB).

a. Unit-level Chunks (Granularity: "unit"):

These chunks represent individual logical code units, such as functions, classes, or smaller code blocks.

- For Python files, the `_extract_python_units` function extracts `ast.FunctionDef`, `ast.AsyncFunctionDef`, and `ast.ClassDef` nodes.

Type your question here...

Send

Ask Questions about the Repository

First, click the "Index Repository" button to make the codebase searchable. Then, you can ask natural-language questions.

Repository:

Index Repository

2. Retrieval (Dual-Stage Retrieval)

The `query_repository_advanced` function implements a dual-stage retrieval strategy, particularly for "HIGH_LEVEL" questions:

a. Stage 1: File-level Retrieval:

- The system first queries the vector database for relevant documents with `"granularity": "file"`. This helps to identify the most relevant files in the repository based on the query.
- This initial broad search helps narrow down the scope to a few promising files.

```
# myapp/embedder.py
# High-level: 2 stage
print("[PIPELINE] Path: HIGH_LEVEL -> stage1 file-level retrieval")
file_docs = _query_chroma_raw(
    collection,
    query_embedding,
    {"repo_url": repo_url, "granularity": "file"}, # <-- Querying file-level chunks
    HIGH_LEVEL_TOP_K_FILES,
```

Type your question here...

Send

Ask Questions about the Repository

First, click the "Index Repository" button to make the codebase searchable. Then, you can ask natural-language questions.

Repository:

Index Repository

```
collection,
query_embedding,
{"repo_url": repo_url, "granularity": "unit", "file_path": fp}, # <-- Querying unit-level chunks within :
HIGH_LEVEL_UNIT_PER_FILE,
)
per_file_units.extend(units_for_file)
```

This dual-level approach aims to balance broad contextual understanding (file-level) with precise detail (unit-level), improving the relevance and quality of retrieved information for complex queries.

Agent Lookup Trace

- Iter 1: status=lookups_fetched requested=5 fetched=5
- Iter 2: Planner decided no lookups needed. Reason: The current context already contains the key functions for dual-level chunking. The initial snippet shows how individual 'units' (first level chunks) are created, likely from `_extract_generic_code_units`. The `_group_units_into_file_chunks` function explicitly describes how these units are then grouped into larger file-level chunks (second level), including handling overlaps. The `_chunk_text` function is also present as a fallback. This provides a comprehensive view of the dual-level chunking process.

Type your question here...

Send

Ask Questions about the Repository

First, click the "Index Repository" button to make the codebase searchable. Then, you can ask natural-language questions.

Repository:

Index Repository

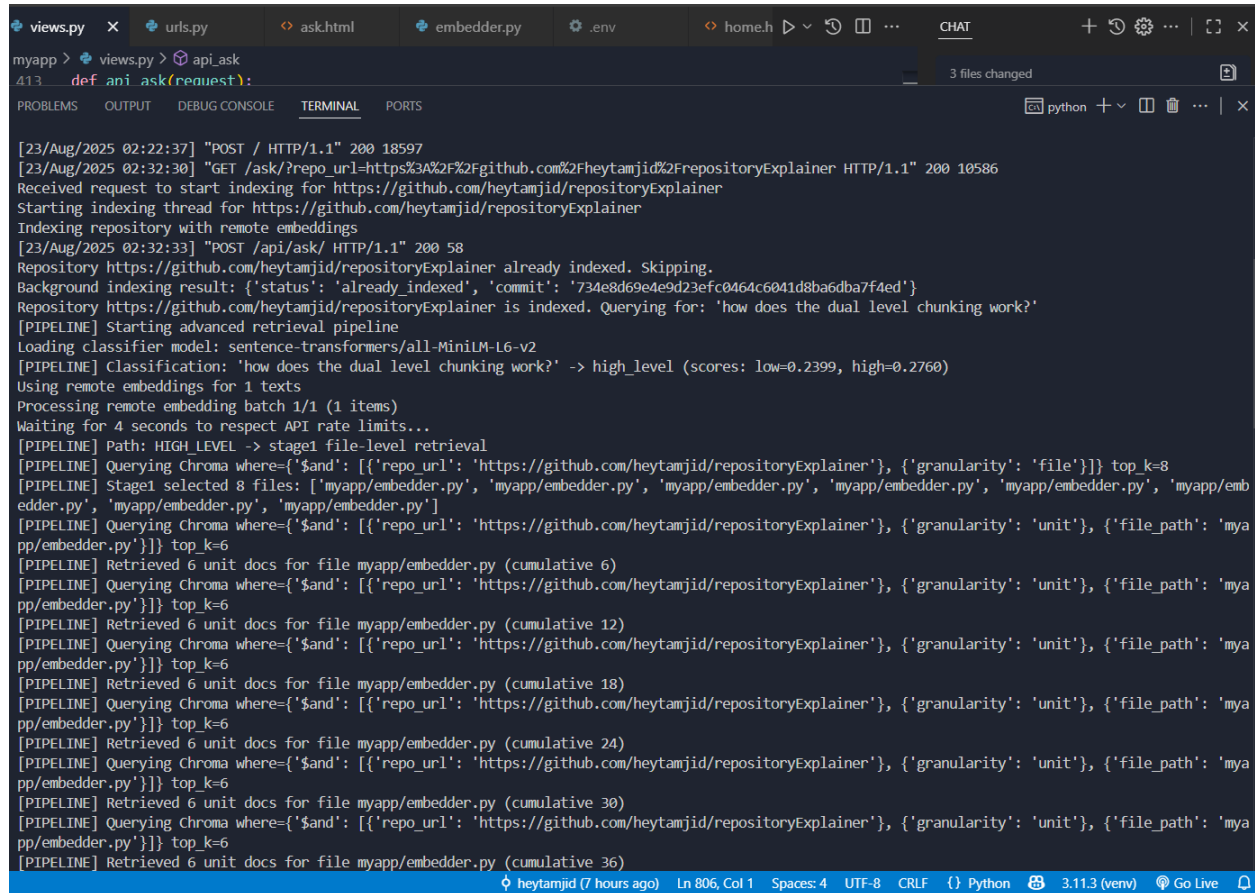
Example of how file-level chunks are created and embedded:

```
# myapp/embedder.py
file_chunks = _group_units_into_file_chunks(
    fp, units if is_python else []
)
for fch_text, fch_idx, fch_start, fch_end in file_chunks:
    doc = (
        f"File Chunk: {rel_path_str}\nChunk Index: {fch_idx}\nLines: {fch_start}-{fch_end}\n" # header
        f"```\n{fch_text}\n```"
    )
    texts_to_embed.append(doc)
    metadata_for_text.append(
        {
            "repo_url": repo_url,
            "commit": commit,
            "file_path": rel_path_str,
            "unit_name": f"file_chunk_{fch_idx}",
            "unit_type": "file",
            "granularity": "file", # <-- Key differentiator
            "start_line": fch_start,
            "end_line": fch_end
        }
    )
```

Type your question here...

Send

9.3 PROJECT SCREENSHOTS OF BACKEND CONSOLE



```

views.py x urls.py ask.html embedder.py .env home.h CHAT
myapp > views.py > api_ask
413 def api_ask(request):
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python + - - - x

[23/Aug/2025 02:22:37] "POST / HTTP/1.1" 200 18597
[23/Aug/2025 02:32:30] "GET /ask/?repo_url=https%3A%2F%2Fgithub.com%2Fheytaamjid%2FrepositoryExplainer HTTP/1.1" 200 10586
Received request to start indexing for https://github.com/heytaamjid/repositoryExplainer
Starting indexing thread for https://github.com/heytaamjid/repositoryExplainer
Indexing repository with remote embeddings
[23/Aug/2025 02:32:33] "POST /api/ask/ HTTP/1.1" 200 58
Repository https://github.com/heytaamjid/repositoryExplainer already indexed. Skipping.
Background indexing result: {'status': 'already_indexed', 'commit': '734e8d69e4e9d23efc0464c6041d8ba6dba7f4ed'}
Repository https://github.com/heytaamjid/repositoryExplainer is indexed. Querying for: 'how does the dual level chunking work?'
[PIPELINE] Starting advanced retrieval pipeline
Loading classifier model: sentence-transformers/all-MiniLM-L6-v2
[PIPELINE] Classification: 'how does the dual level chunking work?' -> high_level (scores: low=0.2399, high=0.2760)
Using remote embeddings for 1 texts
Processing remote embedding batch 1/1 (1 items)
Waiting for 4 seconds to respect API rate limits...
[PIPELINE] Path: HIGH_LEVEL -> stage1 file-level retrieval
[PIPELINE] Querying Chroma where={'$and': [{'repo_url': 'https://github.com/heytaamjid/repositoryExplainer'}, {'granularity': 'file'}]} top_k=8
[PIPELINE] Stage1 selected 8 files: ['myapp/embedder.py', 'myapp/embedder.py', 'myapp/embedder.py', 'myapp/embedder.py', 'myapp/embedder.py', 'myapp/embedder.py', 'myapp/embedder.py', 'myapp/embedder.py']
[PIPELINE] Querying Chroma where={'$and': [{'repo_url': 'https://github.com/heytaamjid/repositoryExplainer'}, {'granularity': 'unit'}, {'file_path': 'myapp/embedder.py'}]} top_k=6
[PIPELINE] Retrieved 6 unit docs for file myapp/embedder.py (cumulative 6)
[PIPELINE] Querying Chroma where={'$and': [{'repo_url': 'https://github.com/heytaamjid/repositoryExplainer'}, {'granularity': 'unit'}, {'file_path': 'myapp/embedder.py'}]} top_k=6
[PIPELINE] Retrieved 6 unit docs for file myapp/embedder.py (cumulative 12)
[PIPELINE] Querying Chroma where={'$and': [{'repo_url': 'https://github.com/heytaamjid/repositoryExplainer'}, {'granularity': 'unit'}, {'file_path': 'myapp/embedder.py'}]} top_k=6
[PIPELINE] Retrieved 6 unit docs for file myapp/embedder.py (cumulative 18)
[PIPELINE] Querying Chroma where={'$and': [{'repo_url': 'https://github.com/heytaamjid/repositoryExplainer'}, {'granularity': 'unit'}, {'file_path': 'myapp/embedder.py'}]} top_k=6
[PIPELINE] Retrieved 6 unit docs for file myapp/embedder.py (cumulative 24)
[PIPELINE] Querying Chroma where={'$and': [{'repo_url': 'https://github.com/heytaamjid/repositoryExplainer'}, {'granularity': 'unit'}, {'file_path': 'myapp/embedder.py'}]} top_k=6
[PIPELINE] Retrieved 6 unit docs for file myapp/embedder.py (cumulative 30)
[PIPELINE] Querying Chroma where={'$and': [{'repo_url': 'https://github.com/heytaamjid/repositoryExplainer'}, {'granularity': 'unit'}, {'file_path': 'myapp/embedder.py'}]} top_k=6
[PIPELINE] Retrieved 6 unit docs for file myapp/embedder.py (cumulative 36)
heytamjid (7 hours ago) Ln 806, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.11.3 (venv) Go Live

```

===== PIPELINE METADATA SUMMARY =====

QUERY: how does the dual level chunking work?

EMBEDDING MODE: remote

CLASSIFICATION: high_level

GROUPED DOC COUNT: 48

TOTAL GROUPED CHARS: 77608

SUMMARY INCLUDED: YES (chars=11960) - sections=4

```
-----
[1] file=myapp/embedder.py lines=712-773 gran=unit size=2074
[2] file=myapp/embedder.py lines=1320-1412 gran=unit size=2072
[3] file=myapp/embedder.py lines=1320-1412 gran=unit size=2070
[4] file=myapp/embedder.py lines=827-1116 gran=unit size=2062
[5] file=myapp/embedder.py lines=712-773 gran=unit size=999
[6] file=myapp/embedder.py lines=776-787 gran=unit size=424
[7] file=myapp/embedder.py lines=712-773 gran=unit size=2074
[8] file=myapp/embedder.py lines=1320-1412 gran=unit size=2072
[9] file=myapp/embedder.py lines=1320-1412 gran=unit size=2070
[10] file=myapp/embedder.py lines=827-1116 gran=unit size=2062
[11] file=myapp/embedder.py lines=712-773 gran=unit size=999
[12] file=myapp/embedder.py lines=776-787 gran=unit size=424
[13] file=myapp/embedder.py lines=712-773 gran=unit size=2074
[14] file=myapp/embedder.py lines=1320-1412 gran=unit size=2072
```

heytaamjid (7 hours ago) Ln 806, C

```
[45] file=myapp/embedder.py lines=1320-1412 gran=unit size=2070
[46] file=myapp/embedder.py lines=827-1116 gran=unit size=2062
[47] file=myapp/embedder.py lines=712-773 gran=unit size=999
[48] file=myapp/embedder.py lines=776-787 gran=unit size=424
```

```
=====
[AGENT] Lookup iteration 1 planning started
[AGENT] Raw plan response (truncated 500 chars): '''json {  "need_lookups": true,  "reason": "The user is asking about 'dual level chunking'. The `REP
O_MAP` shows several functions in `myapp/embedder.py` related to extracting units and chunking text. To understand the 'dual level' aspect, I need to ex
amine the implementation of `index_repository` to see the overall flow, and then look into `_extract_python_units` (or `_extract_generic_code_units`), `_
_group_units_into_file_chunks`, and `_chunk_text` to understand how different levels of chunk
[AGENT] Planner requests 5 lookups: [{ 'type': 'function', 'path': 'myapp/embedder.py', 'name': 'index_repository'}, { 'type': 'function', 'path': 'myapp/
embedder.py', 'name': '_extract_python_units'}, { 'type': 'function', 'path': 'myapp/embedder.py', 'name': '_extract_generic_code_units'}, { 'type': 'func
tion', 'path': 'myapp/embedder.py', 'name': '_group_units_into_file_chunks'}, { 'type': 'function', 'path': 'myapp/embedder.py', 'name': '_chunk_text'}]
[AGENT] Fetched FUNCTION index_repository in myapp/embedder.py lines 827-1116 length=12992
[AGENT] Fetched FUNCTION _extract_python_units in myapp/embedder.py lines 485-508 length=868
[AGENT] Fetched FUNCTION _extract_generic_code_units in myapp/embedder.py lines 591-709 length=3902
[AGENT] Fetched FUNCTION _group_units_into_file_chunks in myapp/embedder.py lines 712-773 length=2725
[AGENT] Fetched FUNCTION _chunk_text in myapp/embedder.py lines 776-787 length=368
[AGENT] Lookup iteration 2 planning started
[AGENT] Raw plan response (truncated 500 chars): '''json {  "need_lookups": false,  "reason": "The current context already contains the key functions fo
r dual-level chunking. The initial snippet shows how individual 'units' (first level chunks) are created, likely from `_extract_generic_code_units`. The
`_group_units_into_file_chunks` function explicitly describes how these units are then grouped into larger file-level chunks (second level), including
handling overlaps. The `_chunk_text` function is also present as a fallback. This provides a
[AGENT] Planner decided no lookups needed. Reason: The current context already contains the key functions for dual-level chunking. The initial snippet s
hows how individual 'units' (first level chunks) are created, likely from `_extract_generic_code_units`. The `_group_units_into_file_chunks` function ex
plicitly describes how these units are then grouped into larger file-level chunks (second level), including handling overlaps. The `_chunk_text` functio
n is also present as a fallback. This provides a comprehensive view of the dual-level chunking process.
[AGENT] TRACE - Iter 1: status=lookups fetched requested=5 fetched=5
[AGENT] TRACE - Iter 2: Planner decided no lookups needed. Reason: The current context already contains the key functions for dual-level chunking. The i
nitial snippet shows how individual 'units' (first level chunks) are created, likely from `_extract_generic_code_units`. The `_group_units_into_file chu
nks` function explicitly describes how these units are then grouped into larger file-level chunks (second level), including handling overlaps. The `_chu
nk_text` function is also present as a fallback. This provides a comprehensive view of the dual-level chunking process.
[23/Aug/2025 02:34:18] "POST /api/ask/ HTTP/1.1" 200 9420
```

heytaamjid (7 hours ago) Ln 806, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.11.3 (venv) Go Live